

Java history and versions

The story of why Java exists, how it evolved, and the version choices that come up in interviews. Factual questions stay tight; the migration and opinion ones are how I would actually talk through them.

1. Who created Java, at which company, and what was it first called?

Java was created by **James Gosling** and his team at **Sun Microsystems** (Oracle acquired Sun in 2010). It was originally called **Oak**, after an oak tree outside Gosling's office. The name was changed because **Oak was already trademarked**, and the team settled on **Java** (after the coffee).

2. Why was Java created, and what problem was it solving?

It started in the early 1990s for **embedded devices** (set-top boxes, appliances), where the pain was that C and C++ code had to be recompiled for every different hardware target. So the problem Java set out to solve was **portability**: run the same program on any device without rewriting or recompiling it. That goal outlived the embedded project and became the whole point of Java once the web arrived.

3. What languages influenced Java?

Mainly **C** and **C++** for the syntax, so it felt familiar to existing developers, but Java **dropped the dangerous parts** (manual memory management, pointers). It also took ideas from **Smalltalk** (the object model, garbage collection) and a few others. The pitch was C++ syntax without the footguns.

4. What were Java's design goals?

The headline goals: **simple** (no pointers or manual memory), **object-oriented**, **portable** and **architecture-neutral** (the WORA goal), **robust** and **secure** (strong typing, bytecode verification, a sandbox), **automatically memory-managed** (garbage collection), and **multithreaded** from the start. Portability and robustness are the two that really defined it.

5. Explain "Write Once, Run Anywhere", and why Java became popular so fast.

WORA means you compile your source once into **bytecode**, and that same bytecode runs on any platform that has a **JVM**, which is the layer that translates bytecode to the local machine. You are not recompiling per operating system; the JVM absorbs the difference. That portability is a big part of why Java took off, and the rest was timing and ecosystem: it arrived with the **web**, it was **free** with a **C-like syntax** developers already knew, it removed the memory bugs that plagued C++, and it shipped a **large standard library** plus strong corporate backing. Portability plus a gentle learning curve plus batteries included.

6. Why does Java release every six months, and what is the difference between LTS and non-LTS?

Since Java 9 (2017), releases follow a **fixed six-month cadence** so features ship steadily instead of being held for years behind one giant release. Most of those are **non-LTS** (short-term): supported only until the next release, meant for early adopters. Every few years one is marked **LTS (Long-Term Support)**, which gets **years of security and bug fixes**, and that is what production systems run on.

LTS version	Year
Java 8	2014
Java 11	2018
Java 17	2021
Java 21	2023
Java 25	2025

7. Which Java version would you choose for a new enterprise application, and why do companies stay on LTS?

For anything new I would start on the **latest LTS** (Java 21, or 25 now that it is out), not a non-LTS release. The reason is simple: an enterprise app lives for years, and I want **years of security patches** behind it without a forced upgrade every six months. That is also why companies stay on LTS in general, they are trading the newest features for **stability and a long support window**, because an unsupported runtime in production is a security and audit problem. Picking up a non-LTS release would mean re-qualifying the JDK every six months, which no ops team wants.

8. OpenJDK, Oracle JDK, and why organisations moved to OpenJDK.

OpenJDK is the **free, open-source reference implementation** of Java. **Oracle JDK** is Oracle's build of it, historically with a commercial licence for production use. The two are **functionally near-identical today** (Oracle's JDK is largely OpenJDK plus their branding and commercial support). The migration happened because of **licensing**: Oracle changed its terms so that using Oracle JDK in production could require a **paid subscription**, so many organisations moved to OpenJDK builds (from Adoptium/Temurin, Amazon Corretto, Azul, and others) to **avoid the licensing cost and risk** while getting the same runtime.

9. Why is Java 8 considered one of the biggest releases, and what did each headline feature bring?

Because it brought **functional programming into Java** and changed how everyday code is written. The headline features:

- **Lambda expressions**: anonymous functions in a compact syntax. They solved the **verbosity of anonymous inner classes**, so passing behaviour around (a comparator, a callback) went from many lines to one.
- **Functional interfaces**: an interface with a single abstract method (`Runnable` , `Comparator`), which is what a lambda actually implements.
- **Method references**: shorthand for a lambda that just calls one method (`String::length`).
- **Stream API**: declarative processing of collections (filter, map, reduce) instead of manual loops, with easy parallelism.
- **Optional**: a container for a maybe-absent value, to reduce `NullPointerException` .
- **Default methods**: methods with a body inside an interface, which let interfaces evolve without breaking existing implementers.
- **New Date and Time API (`java.time`)**: an immutable, sane replacement for the old broken `Date` and `Calendar` .

- **CompletableFuture** : composable asynchronous programming, chaining async steps without callback nesting.
- **Collection factory methods** (`List.of()` , `Set.of()` , `Map.of()`): quick immutable collections in one call.

(The deeper mechanics of streams, lambdas, and `Optional` live in the Java 8 features sheet; here it is about why the release mattered.)

10. Why is Java 21 considered a major release, and what did it bring?

Because it is the LTS that made **lightweight concurrency** real. The headline features:

- **Virtual threads**: extremely cheap threads managed by the JVM, so you can run millions of them. They let you write simple **blocking, thread-per-request** code that scales like async, without the callback complexity. This is the big one for servers.
- **Structured concurrency** (preview): treats a group of related concurrent tasks as **one unit**, so they start and finish together and errors and cancellation propagate cleanly, instead of leaking stray threads.
- **Scoped values** (preview): a safer, immutable alternative to `ThreadLocal` for sharing data within a task, which fits virtual threads well.
- **Record patterns**: destructure a record directly in a pattern match (`case Point(int x, int y)`), which pairs with pattern matching for `switch` .

11. What are the significant features introduced after Java 21?

The releases after 21 (22, 23, 24, and the Java 25 LTS in 2025) mostly **finalised things that were in preview** and added smaller language conveniences: **stream gatherers** (custom intermediate stream operations), continued work on **structured concurrency** and **scoped values**, **flexible constructor bodies**, and **module import declarations**. **Java 25** is the next LTS and rolls these up. I would confirm the exact per-version list against the release notes rather than trust memory, since this moves fast.

12. Why is Java known for strong backward compatibility, and how is it maintained?

Because **old code generally keeps running on new JVMs**: bytecode compiled years ago still executes, and source written against an old API still compiles. It was a deliberate promise so enterprises could upgrade the runtime without rewriting applications. It is maintained by **not removing public APIs lightly** (features get **deprecated** and left in place for years before any removal), keeping the **bytecode format stable and versioned**, and putting new behaviour behind **new APIs** rather than changing the old ones. The result is that the safe default is that your code still works.

13. If Java is backward compatible, why do some applications still break after a JDK upgrade?

Because backward compatibility is **strong, not absolute**. The usual causes:

- **Removed or long-deprecated APIs** finally get deleted (whole modules like the Java EE ones, CORBA, were removed after Java 8).
- **Access to JDK internals**: code (or a library) reached into `sun.*` or used deep reflection, which newer modular JDKs (Java 9+ with the module system) now block.
- **Third-party libraries and build tools** that were never updated for the new JDK, so the break is really in a dependency, not your code.
- **Behavioural changes**: default garbage collector, encoding, or TLS defaults changed between versions.

So the runtime keeps the contract, but old internal-API hacks and stale dependencies are what actually bite.

14. Your application runs on Java 8. How would you plan a migration to Java 21?

The way I would approach it: treat it as **incremental and evidence-driven**, not a big-bang jump. First I would **inventory the dependencies** and check each one supports Java 21, because outdated libraries are the most common blocker. Then I would **build and run the test suite on 21** early to surface breaks, and lean on tools to assess compatibility: the `jdeps` tool to find internal-API and removed-module usage, and `jdeprscan` to catch deprecated APIs. I would go through an **intermediate LTS (Java 17)** rather than straight to 21 if the jump throws up too much at once, fix the module and dependency issues, and only then move up. Runtime changes matter too, so I would **re-check GC and memory behaviour** under load, since defaults changed. The theme is: let the compiler and tests tell you what breaks, upgrade dependencies first, and move in steps.

15. Which Java 21 features would give the biggest benefit for a microservices application?

Virtual threads, easily. Microservices spend most of their time **waiting on I/O** (other services, databases), and virtual threads let each request own a cheap blocking thread instead of sharing a small pool, so you get **high concurrency with simple, readable blocking code** and no reactive/callback complexity. **Structured concurrency** helps too, because a service often fans out to several downstream calls, and it lets you run those as one unit with clean cancellation and error handling. Those two together are the reason I would push a service-heavy stack toward 21.

16. Have you hit compatibility issues after a JDK upgrade, and how did you resolve them?

The honest pattern is that the breaks are almost always in **dependencies and internal-API access, not my own business logic**. A typical one: a library reflecting into JDK internals that the module system now blocks, which showed up as an `IllegalAccessError` at runtime rather than a compile error. The fix was to **upgrade the library** to a JDK-compatible version, and where none existed, replace it. Removed modules (the Java EE APIs dropped after Java 8) are another common one, resolved by **adding them back as explicit dependencies**. The way I work through it is to run the tests on the new JDK, read the actual error, and trace it to the offending dependency rather than guessing. *(Adapt to a real upgrade you did.)*

17. If you could remove one Java feature, what would it be and why?

I would point at **checked exceptions**. The intent was good, force callers to handle failure, but in practice they push people toward **swallowing exceptions or wrapping everything in RuntimeException**, and they do not compose well with lambdas and streams, which is why newer languages dropped the idea. So the feature meant to improve error handling often makes it worse. *(This is an opinion question, so any defensible pick with a real reason works; the point is to show judgment.)*

18. Which modern Java features would you encourage teams to adopt, and which would you hold back on?

I would **encourage** the ones that make code clearer and safer with little downside: **records** for data carriers, **Optional** at API boundaries, **streams** for readable transformations, **var** for obvious local types, and **virtual threads** for I/O-bound services. I would **hold back initially** on anything still in **preview** (structured concurrency, scoped values), because the API can change between versions and I do not want production code chasing it, and I would discourage **overusing streams** for complex logic where a plain loop reads better. The rule I would give a team is: adopt the finalised, clarity-improving features freely, but wait for preview features to stabilise before betting on them.